

# O&G · 5 — A physics-informed neural network for pressure-transient analysis ⇨

`04_well_test_history_matching.jl` used a closed-form *approximation* to the diffusivity equation. Here we solve the actual dimensionless radial diffusivity PDE directly with a physics-informed neural network, the same technique as `05_physics_informed_neural_networks.jl`, applied to a bounded (closed outer-boundary) reservoir:

$$\frac{\partial p_D}{\partial t_D} = \frac{1}{r_D} \frac{\partial}{\partial r_D} \left( r_D \frac{\partial p_D}{\partial r_D} \right), \quad r_D \in [1, r_{eD}]$$

with a constant-rate inner boundary (Neumann), a no-flow outer boundary (Neumann), and zero initial pressure depletion:

$$\left. \frac{\partial p_D}{\partial r_D} \right|_{r_D=1} = -1, \quad \left. \frac{\partial p_D}{\partial r_D} \right|_{r_D=r_{eD}} = 0, \quad p_D(r_D, 0) = 0$$
$$p_D(1, t_D)$$

— the dimensionless pressure at the wellbore — is exactly the quantity a pressure gauge records during a well test.

```
1 using NeuralPDE, Lux, ModelingToolkit, Optimization, OptimizationOptimisers, DomainSets
   , Random, Plots
```

## 1. Symbolic PDE specification ⇨

```
► [r, t]
```

```
1 ModelingToolkit.@parameters r t
```

⚠ Attempting to print a symbolic expression in a Pluto notebook. Please run ``import LaTeXify`` to enable pretty-printing of symbolic expressions. This warning will only display once.

```
► [p★]
```

```
1 ModelingToolkit.@variables p(..)
```

```
► [VarDomainPairing(r, 1.0 .. 20.0), VarDomainPairing(t, 0.0 .. 40.0)]
```

```
1 begin
2   reD = 20.0      # dimensionless drainage radius (re / rw)
3   tDmax = 40.0
4
5   Dt = ModelingToolkit.Differential(t)
6   Dr = ModelingToolkit.Differential(r)
7   Drr = Dr^2
8
9   eq = Dt(p(r, t)) ~ Drr(p(r, t)) + (1 / r) * Dr(p(r, t))
10
11   bcs = [p(r, 0.0) ~ 0.0,
12          Dr(p(1.0, t)) ~ -1.0,
13          Dr(p(reD, t)) ~ 0.0]
14
15   domains = [r ∈ Interval(1.0, reD), t ∈ Interval(0.0, tDmax)]
16 end
```

## 2. The network ansatz and training problem ↻

```
Chain(
  layer_1 = Dense(2 => 24, tanh),      # 72 parameters
  layer_2 = Dense(24 => 24, tanh),     # 600 parameters
  layer_3 = Dense(24 => 1),            # 25 parameters
) # Total: 697 parameters,
# plus 0 states.
```

```
1 begin
2   rng_og5 = Random.default_rng()
3   Random.seed!(rng_og5, 2)
4   pde_chain = Chain(Dense(2, 24, tanh), Dense(24, 24, tanh), Dense(24, 1))
5 end
```

```
pde_discretization =
```

```
► PhysicsInformedNN(Chain{@NamedTuple{layer_1::Dense{typeof(tanh)}, Int64, Int64, Nothing, Noth
```

```
1 pde_discretization = PhysicsInformedNN(pde_chain, GridTraining(0.25))
```

```
OptimizationProblem. In-place: true
```

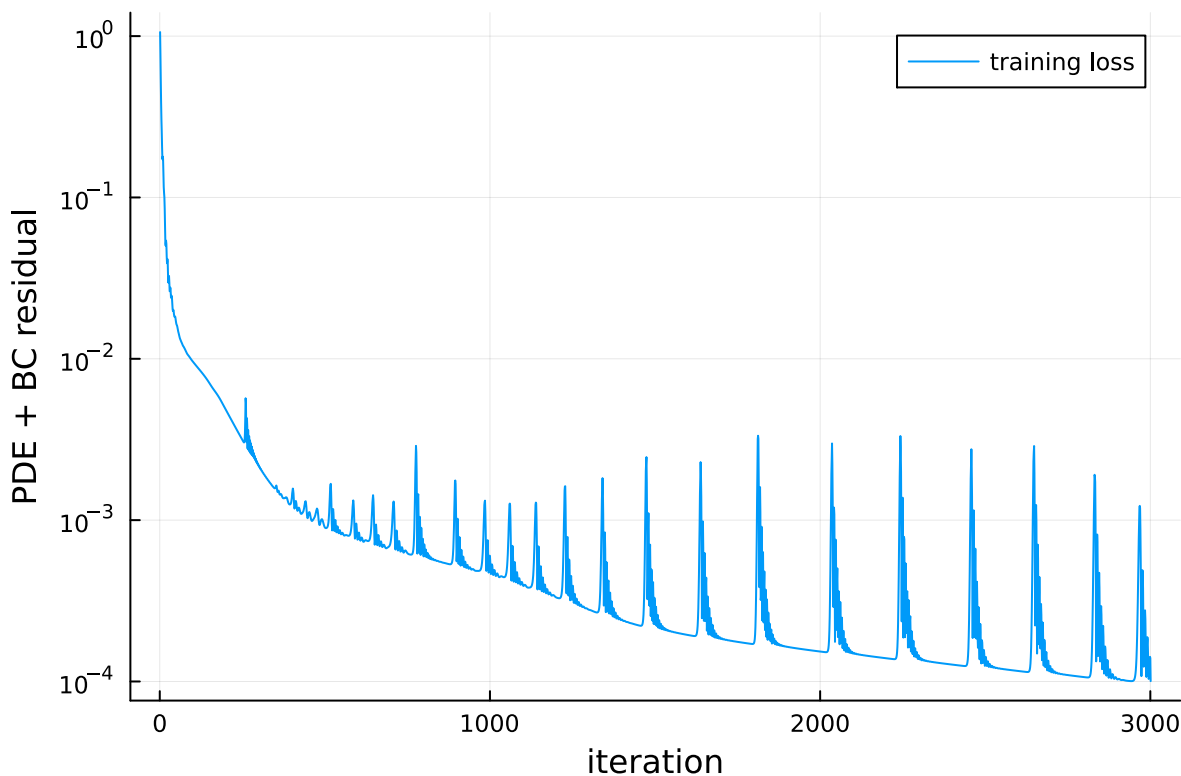
```
u0: ComponentVector{Float64}{layer_1 = (weight = [1.3821232318878174 1.5619776248931885; -0.:
```

```
1 begin
2   @named pressure_pde_system = PDESystem(eq, bcs, domains, [r, t], [p(r, t)])
3   prob_pde = discretize(pressure_pde_system, pde_discretization)
4 end
```

### 3. Train ⇌

► SciMLBase.OptimizationSolution{Float64, 1, ComponentArrays.ComponentVector{Float64, Vector{Float64}}

```
1 begin
2     pde_losses = Float64[]
3     pde_callback = function (state, loss)
4         push!(pde_losses, loss)
5         return false
6     end
7
8     res_pde = Optimization.solve(prob_pde, OptimizationOptimisers.Adam(0.01);
9     callback=pde_callback, maxiters=3000)
10 end
```



```
1 plot(pde_losses; xlabel="iteration", ylabel="PDE + BC residual", yscale=:log10,
2 label="training loss")
```

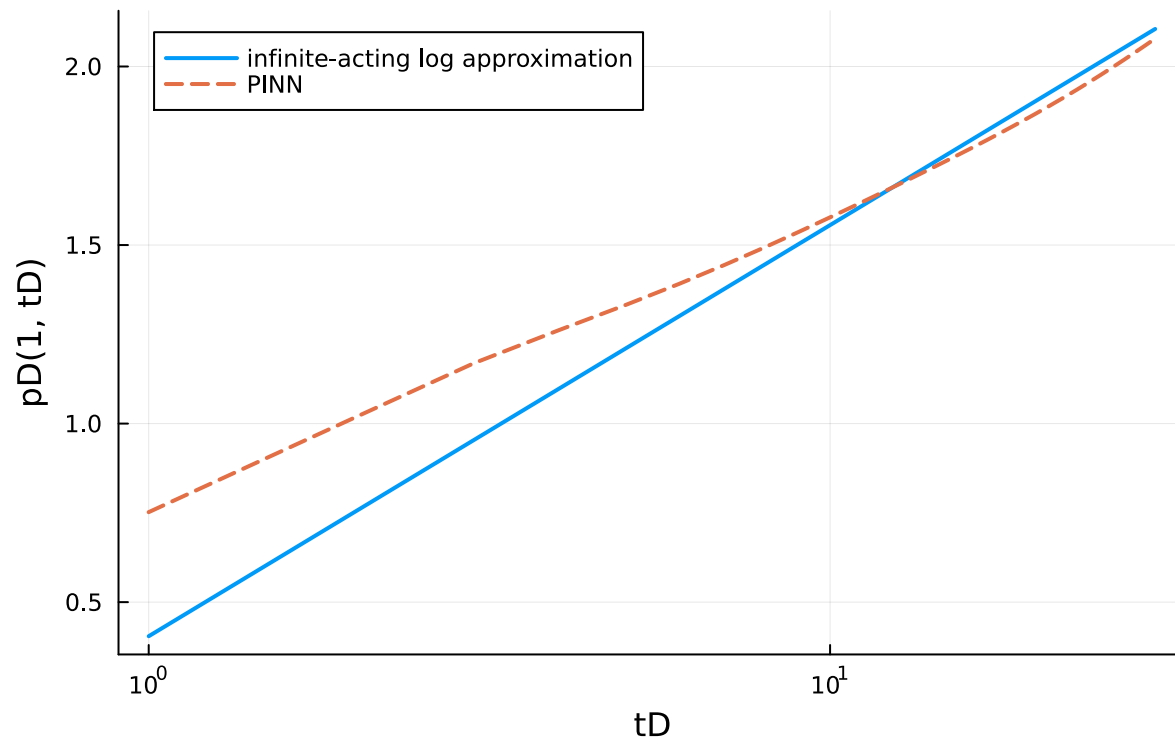
### 4. The wellbore pressure response — and a sanity check against the early-time analytical solution ⇌

Before the pressure disturbance reaches the outer boundary, the well behaves as if the reservoir were infinite-acting, and  $p_D(1, t_D)$  should follow the well-known log approximation

$$p_D(1, t_D) \approx \frac{1}{2} [\ln(t_D) + 0.80907]$$

This reservoir is small ( $r_{eD} = 20$ ), so pseudo-steady-state sets in early — around  $t_D \approx 0.1 r_{eD}^2 = 40$ , right at the edge of the domain we trained over. Expect the PINN curve to track the infinite-acting line only at the start of the range and bend away from it as  $t_D$  climbs toward that boundary-dominated regime — that bend is the reservoir feeling its outer edge, not a training artifact.

## Wellbore pressure response (PINN vs. analytical early-time solution)



```

1 begin
2   phi_pde = pde_discretization.phi
3   final_pde_ps = res_pde.u
4
5   t_grid = 1.0:1.0:30.0
6   pD_wellbore = [first(phi_pde([1.0, ti], final_pde_ps)) for ti in t_grid]
7   pD_analytical = [0.5 * (log(ti) + 0.80907) for ti in t_grid]
8
9   plot(t_grid, pD_analytical; xscale=:log10, label="infinite-acting log
approximation", lw=2, xlabel="tD", ylabel="pD(1, tD)")
10  plot!(t_grid, pD_wellbore; xscale=:log10, label="PINN", lw=2, ls=:dash,
11         title="Wellbore pressure response (PINN vs. analytical early-time solution)")
12 end

```

## Takeaways ↻

- The Neumann inner-boundary condition — the actual physical statement "the well produces at a constant rate" — is passed to `NeuralPDE.jl` as literally  $\partial p_D / \partial r_D|_{r_D=1} = -1$ , no discretized flux calculation required.
- At early times the PINN's wellbore response tracks the classical infinite-acting log approximation; as  $t_D$  approaches the pseudo-steady-state threshold ( $\approx 0.1 r_{eD}^2$ ) it bends away — the same

**infinite-acting** → **transition** → **boundary-dominated flow** signature every well test interpreter looks for, reproduced here from the PDE alone rather than assumed.

- Because the whole reservoir domain was solved in one mesh-free network, extracting the pressure at *any* radius (not just the wellbore) is a free  $\phi(r, t, p)$  evaluation — useful for interference-test analysis at an offset observation well.